



Phase 1 Project Requirements: Entra ID SecOps MCP Server

1. Objective

Conduct a brief market scan of existing open-source MCP repositories to ensure a baseline framework doesn't already exist. Once validated, build a lightweight, containerized Model Context Protocol (MCP) server that exposes key Microsoft Graph API log endpoints as executable tools.

This delivery represents Phase 1 of a broader automation initiative; the server will ultimately act as the primary telemetry interface for a downstream AI agent designed to autonomously orchestrate and accelerate incident triage. As the final task of this phase, research secure architectures and communication methods to expose the MCP server so remote clients can safely connect to it in future stages.

2. Functional Requirements (The Tools)

Expose specific, high-value API endpoints to the LLM. For all tools, implement aggressive server-side data truncation so that massive Microsoft JSON payloads are stripped down to core security indicators before reaching the LLM's finite context window.

- **Tool 1: get_user_signins**
 - *Purpose:* Fetches recent sign-in logs for a specific User Principal Name (UPN) to investigate login blocks or anomalies.
 - *Suggested Output:* Timestamp, App Name, IP Address, Geo-location, Login Status (Success/Failure), and Failure Reason.
- **Tool 2: get_risky_users**
 - *Purpose:* Pulls the tenant's top high-risk accounts flagged by Entra Identity Protection.
 - *Suggested Output:* User Principal Name, Risk Level, Risk State, and Last Updated Timestamp.
- **Tool 3: get_directory_audits**
 - *Purpose:* Retrieves recent administrative changes to identify configuration drift or unauthorized modifications.
 - *Suggested Output:* Activity Date, Activity Name, Initiated By (User/App), Target Resource, and Result Status.
- **Bonus Tooling:**
 - Highly encouraged to identify and implement 1 or 2 additional high-value tools based on standard SecOps playbooks (e.g., querying active Conditional Access policies or pulling specific group memberships).



3. Technical & Security Requirements

Core Mandates (Non-Negotiable)

- Language: Must use Python for the server implementation.
- Packaging: Must use Docker to containerize the application.
- Secret Isolation: Strict prohibition of hardcoded credentials anywhere in the codebase or Dockerfile. All runtime credentials must be passed dynamically via host-mapped environment configuration files (e.g., using --env-file flags).

Design Suggestions (Flexible)

- Framework: Consider using the official mcp Python SDK (such as FastMCP) to streamline rapid prototyping via docstring-driven tool schemas.
- Authentication: Utilizing the azure-identity library (like ClientSecretCredential) is recommended to manage clean OAuth 2.0 background token rotation cycles.
- Entra ID Scopes: Target least-privilege permissions based on the chosen endpoints. Typical candidate permissions include AuditLog.Read.All, IdentityRiskyUser.Read.All, and Directory.Read.All.
- Execution Safety: Omitting Docker TTY allocation (avoiding the -t parameter during execution) is recommended to ensure standard output streams remain pure, uncolored JSON.

4. Definition of Done

Phase 1 is officially complete when the following criteria are met:

- The Container: A functional Docker image (the MCP server) that successfully establishes a connection with Microsoft Graph and returns parsed log outputs.
- The Guide: Accompanying documentation explicitly detailing how to configure the target LLM client (e.g., Claude Desktop, Cursor) to interface with the container, alongside instructions for secure local secret management.
- The Research: A technical brief summarizing researched already available MCPs for Microsoft and security patterns (e.g., reverse proxies, SSE configurations, or secure tunnels) for safely exposing the server.
- The Live Test: Successful verification where a natural language prompt correctly isolates an authentication failure or risk status via the LLM chat window in under 10 seconds.